MODELOS DE LINGUAGEM E GENERATIVOS

# Modelos de Linguagem e Generativos

AULA 3 • REDES NEURAIS & TENSORFLOW

# Roteiro da Aula: A Infraestrutura para Deep Learning

---

1. **Módulo 1: Fundamentos de Redes Neurais** ← Link
Perceptron, funções de ativação (Sigmoid, Tanh, ReLU), o problema XOR e arquiteturas MLP com Backpropagation.

2. **Módulo 2: O Framework TensorFlow & Tensores** ← Link
Estrutura de dados multidimensional (Tensores), alocação em GPU, operações matriciais vetorizadas e o ecossistema Keras.

3. **Módulo 3: Programação Diferenciável & AutoGrad** ← Link
Cálculo automático de gradientes com tf.GradientTape(), grafos computacionais, modo Eager e depuração.

4. **Módulo 4: Construção de Modelos no Keras** ← Link
Comparação prática entre Modelo Sequencial, API Funcional e Subclassing de tf.keras.Model.

5. **Módulo 5: Treinamento Customizado & Prática Word2Vec** ← Link
Implementação do Skip-Gram com Negative Sampling do zero no TensorFlow, métricas e visualização no TensorBoard.

# O Neurônio Artificial (Perceptron) e Ativações

## Estrutura do Perceptron

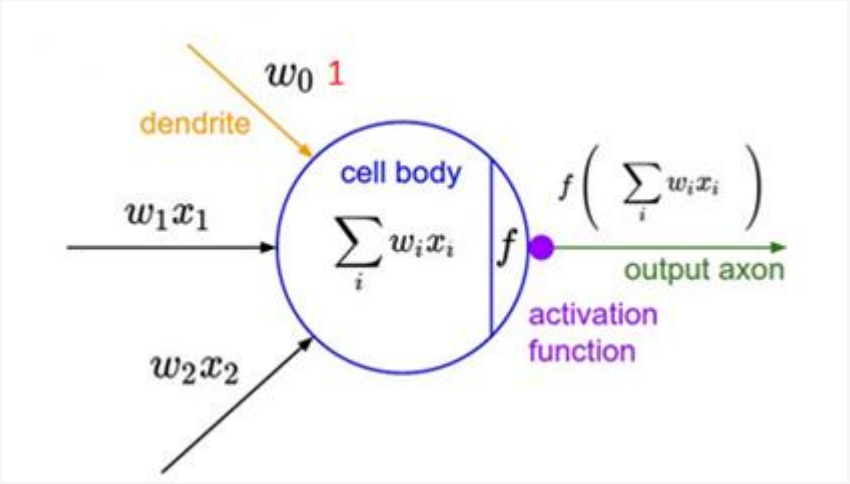
Combinação Linear +

$f(X) = \text{activation}(w_0 + w_1x_1 + w_2x_2 + ... + w_nx_n)$

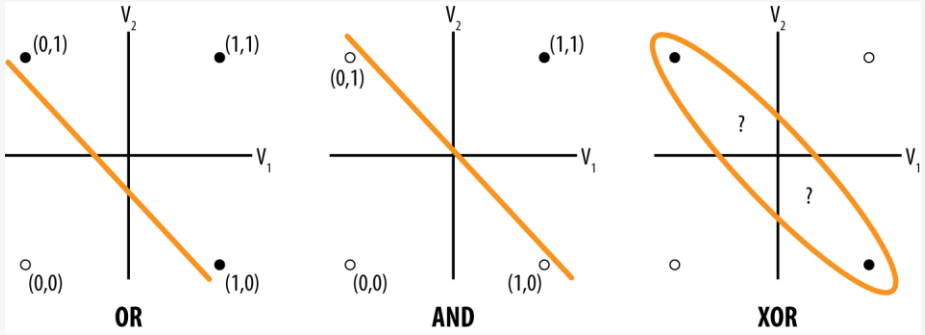
O aprendizado dos pesos (w) é um problema de otimização para minimizar o erro de predição.

Funções de Ativação Tradicionais:

- Sigmoid / Logística
- Tanh (Tangente Hiperbólica
- ReLU (Rectified Linear Unit): f(x) = max(0, x).



# O Neurônio Artificial (Perceptron) e Ativações




## Limitação do Perceptron: O Problema XOR

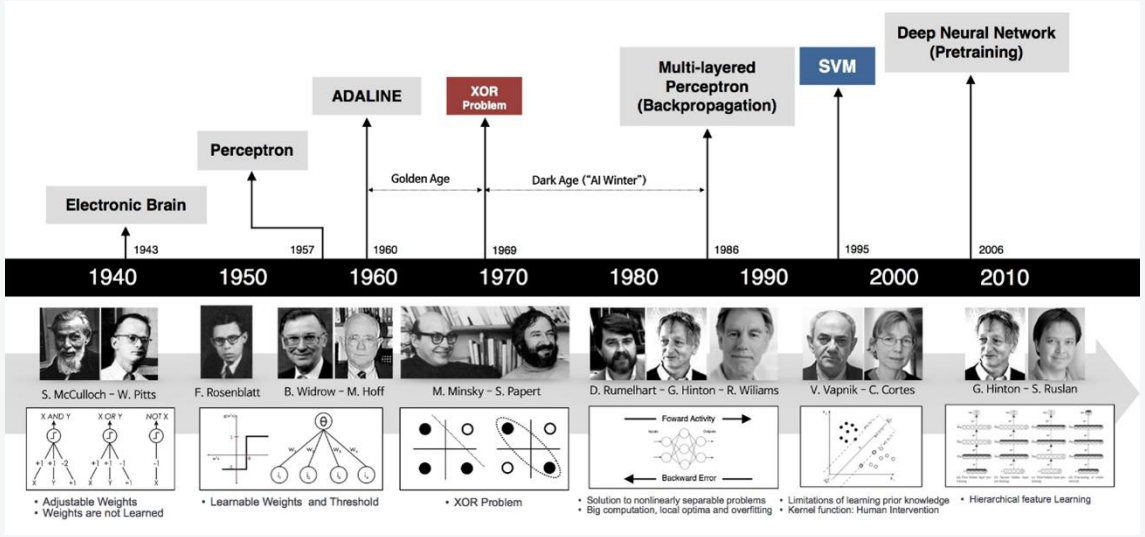
O Perceptron de camada única aprende apenas hiperplanos lineares.

A função XOR NÃO é linearmente separável (Score = 0.5 com 1 neurônio).

Conectar múltiplos perceptrons em camadas ocultas (Multilayer Perceptron - MLP).

O surgimento do algoritmo de Backpropagation para ajustar pesos em múltiplas camadas simultaneamente.

# O Neurônio Artificial (Perceptron) e Ativações




## Limitação do Perceptron: O Problema XOR

O Perceptron de camada única aprende apenas hiperplanos lineares.

A função XOR NÃO é linearmente separável (Score = 0.5 com 1 neurônio).

Conectar múltiplos perceptrons em camadas ocultas (Multilayer Perceptron - MLP).

O surgimento do algoritmo de Backpropagation para ajustar pesos em múltiplas camadas simultaneamente.

# Redes Multicamada (MLP) e Backpropagation

## Treinamento em Duas Etapas

### 1. Forward Pass (Passe para Frente)

Os dados de entrada passam sequencialmente camada por camada até gerar a saída predita (ŷ) e o cálculo da Loss Function (L).

### 2. Backward Pass (Retropropagação do Erro)

Através da Regra da Cadeia (Chain Rule), calcula-se a derivada do erro em relação a cada peso (∂L/∂w) da camada de saída até a entrada.

Otimizadores Avançados

- SGD / Mini-Batch SGD
- Adam (Adaptive Moment Estimation

```
# Baseline MLP no Scikit-Learn (T2)
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split

# Exemplo Baseline com Scikit-Learn (Notebook T2)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, random_state=123
)

mlp = MLPClassifier(
    hidden_layer_sizes=(8, 8),
    activation='relu',
    solver='adam',
    max_iter=500
)
mlp.fit(X_train, y_train)

# Saída do Modelo:
# Acurácia no teste (Biopsy/Cancer): 0.957 (95.7%)
```

Acesse: https://playground.tensorflow.org/

# Tensores: A Estrutura de Dados Fundamental

## Hierarquia e Ranks de Tensores

Definição: Matrizes multidimensionais com um tipo uniforme (dtype) otimizadas para processamento paralelo.

Ranks e Dimensões

- Rank-0 (Escalar): Um único valor sem eixos. Ex: tf.constant(3.14)
- Rank-1 (Vetor): 1 eixo. Ex: tf.constant([2.0, 3.0, 4.0])
- Rank-2 (Matriz): 2 eixos (linhas x colunas). Ex: shape=(3, 2)
- Rank-4 (Lotes de Imagens/Dados): Ex: shape=(Batch, Height, Width, Channels) = (32, 28, 28, 3)

Constantes vs. Variáveis:

- tf.constant(): Imutável.
- tf.Variable(): Mutável via .assign(). Guarda pesos treináveis.

```
# Operações com Tensores no TF (T3)
import tensorflow as tf
import numpy as np

# Criando Tensores no TensorFlow 2.x
scalar = tf.constant(2.7, dtype=tf.float32)
matrix = tf.constant([[1.0, 2.0], [3.0, 4.0]])
var_tensor = tf.Variable([1.0, 2.0, 3.0])

# Mutabilidade de Variáveis
var_tensor[0].assign(9.0)

print('Shape:', matrix.shape)  # (2, 2)
print('Dtype:', matrix.dtype)  # float32
print('Numpy:', matrix.numpy())

# Saída:
# tf.Tensor([[1. 2.] [3. 4.]], shape=(2, 2))
```

# Processamento em GPU e Operações Matriciais

## Aceleração Hardware e Vetorização

Por que usar Frameworks de Deep Learning?

- Eliminação de laços Python: Operações executadas diretamente sobre blocos de memória contígua em C++/CUDA.
- Transparência de Dispositivo: Tensores são alocados automaticamente na GPU sem necessidade de reescrever o código.

## Cuidados

- Multiplicação de Matrizes: Use tf.matmul(a, b) ou a @ b.
- Cuidado com '*': O operador * faz multiplicação elemento a elemento (Hadamard / Broadcast), NÃO produto de matrizes!
- Compatibilidade de Tipos: O TensorFlow não faz conversão implícita entre int32 e float32.

```
# Alocação Transparente em GPU (T3)
import tensorflow as tf

# Log de alocação de dispositivo
tf.debugging.set_log_device_placement(True)

a = tf.constant([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
b = tf.constant([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])

# Produto Matricial (2x3) @ (3x2) -> (2x2)
c = tf.matmul(a, b)  # ou a @ b

# Saída do Log de Alocação:
# Executing op MatMul in device /device:GPU:0
# tf.Tensor([[22. 28.] [49. 64.]], shape=(2, 2))
```

# Diferenciação Automática com tf.GradientTape

## O Conceito de AutoGrad

**CHAVE das Redes Neurais Modernas**

- Permite calcular derivadas exatas de qualquer expressão matemática sem necessidade de derivação manual.

Como funciona o tf.GradientTape():

- O contexto 'with tf.GradientTape() as tape:' grava todas as operações executadas sobre variáveis com gradiente ativado.
- O método tape.gradient(target, sources) aplica a regra da cadeia de trás para frente.

Características Relevantes:

- Liberação de Memória: Por padrão, a fita é descartada após a 1ª chamada ao .gradient(). Use persistent=True para reusá-la.

```
# Cálculo Automático de Gradientes (T3)
import tensorflow as tf

# Exemplo 1: y = x^2  =>  dy/dx = 2x
x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = x * x

dy_dx = tape.gradient(y, x)
print('dy/dx em x=3:', dy_dx.numpy())  # 6.0

# Exemplo 2: y = tanh(x)
x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = tf.nn.tanh(x)
dy_dx = tape.gradient(y, x)
print('dtanh/dx:', dy_dx.numpy())      # 0.00986
```

# Grafos Computacionais e Eager Execution

## Eager Execution vs. Grafos Estáticos

Eager Execution (Modo Imperativo - Padrão TF 2.x):

- Operações são avaliadas imediatamente ao serem chamadas no Python.
- Facilita a depuração, impressão e inspeção do código linha a linha.

Grafos de Execução (@tf.function / AutoGraph):

- Converte funções Python em um Grafo de Fluxo de Dados otimizado em C++.
- Nó = Operação matemática; Aresta = Tensor.
- Vantagens: Otimização de memória, fusão de operações e fácil implantação em servidores/mobile (TensorBoard).

```
# Compilação de Grafo com @tf.function (T3)
import tensorflow as tf

# Função rastreada com @tf.function
@tf.function
def minha_funcao(x, y):
    return tf.pow(x, 2) + tf.pow(y, 3) + x * y - 10

x = tf.Variable([3], dtype=tf.int32)
y = tf.Variable([2], dtype=tf.int32)

z = minha_funcao(x, y)
print('Resultado z:', z.numpy())  # [13]

# O TF compila e otimiza o grafo automaticamente
# para ser executado em C++ ou exportado no TensorBoard
```

# As 3 Formas de Construir Modelos no Keras

## 1. Modelo Sequencial

Pilha linear simples de camadas. Ideal para 90% das redes feed-forward tradicionais com 1 entrada e 1 saída.

**Sintaxe Básica:**

```
keras.Sequential([layers.Dense(...)])
```

## 2. API Funcional

Estruturação flexível tipo grafo. Suporta múltiplas entradas, conexões residuais (skip connections) e saídas ramificadas.

**Sintaxe Básica:**

```
inputs = Input()
x = Dense(...)(inputs)
model = Model(inputs, x)
```

## 3. Subclassing (POO)

Orientação a objetos total derivando de tf.keras.Model. Controle absoluto do Forward Pass no método call(). Usado nos Transformers/LLMs.

**Sintaxe Básica:**

```
class MyModel(tf.keras.Model):
    def call(self, inputs): ...
```

# Modelo Sequencial para Classificação

## Etapas de Construção e Compilação

1. Definindo as Camadas (layers.Dense)
2. Compilando o Modelo (model.compile)

```
# Código Keras Sequencial (T4)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Declaração do Modelo Sequencial
model = keras.Sequential([
    layers.Dense(8, activation='sigmoid',
input_shape=[9]),
    layers.Dense(1, activation='sigmoid')
])

# Compilação com Loss e Otimizador
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['binary_accuracy']
)
```

# Treinamento, Callbacks e Validação

## Parâmetros e Early Stopping

Parâmetros principais no model.fit():

- epochs: Número de passagens completas pelo dataset.
- batch_size: Amostras por atualização de gradiente (default = 32). Afeta diretamente a velocidade do treino.
- validation_split: Fração reservada para validação (ex: 0.3).

Interrupção Precoce (EarlyStopping):

- Evita o overfitting monitorando a val_loss.
- patience=10: Aguarda 10 épocas sem melhoria antes de parar.
- restore_best_weights=True: Restaura automaticamente os melhores pesos.

```
# Execução e Histórico com Fit (T4)
# Configurando Callback de Parada Precoce
early_stopping = keras.callbacks.EarlyStopping(
    patience=10,
    min_delta=0.001,
    restore_best_weights=True
)

# Executando o Treinamento
history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=100,
    validation_split=0.3,
    callbacks=[early_stopping],
    verbose=0
)

# Saída Histórica:
# Best Validation Accuracy: 0.957 (95.7%)
```

# Construção com a API Funcional do Keras

## Conectando Camadas como Funções

Vantagens da API Funcional

- Cada camada é tratada explicitamente como uma função que recebe um tensor e retorna outro tensor.
- Permite reutilização de camadas, conexões não sequenciais e múltiplas saídas.
- Inspeção de Parâmetros com model.summary()

```
# Exemplo de API Funcional (T4)
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Definição Explícita de Fluxo de Tensores
inputs = Input(shape=(9,))           # Tensor de Entrada
out1 = Dense(8, activation='sigmoid')(inputs)
out2 = Dense(1, activation='sigmoid')(out1)

model = Model(inputs=inputs, outputs=out2)
model.summary()

# Output do Summary:
# Layer (type)         Output Shape     Param #
# input_1 (Input)      [(None, 9)]      0
# dense_1 (Dense)      (None, 8)        80
# dense_2 (Dense)      (None, 1)        9
# Total params: 89
```

# Subclassing de Modelos (tf.keras.Model)

## Orientação a Objetos no Keras

A Abordagem dos Pesquisadores e do Hugging Face:

- herda diretamente de tf.keras.Model.
- Método __init__(): Instancia as camadas e sub-blocos da rede.
- Método call(): Define imperativamente o Forward Pass (como os tensores fluem).

Por que usar Subclassing?

- Permite incluir laços for, condicionais e estruturas dinâmicas dentro do Forward Pass.
- É o padrão universal utilizado na implementação de Transformers (BERT, GPT, Llama).

```
# Classe Subclassed de Modelo (T4)
import tensorflow as tf
from tensorflow.keras import layers, Model

# Criando Classe do Modelo Customizado
class MyModel(Model):
    def __init__(self, **kwargs):
        super(MyModel, self).__init__(**kwargs)
        self.dense1 = layers.Dense(8,
activation='sigmoid')
        self.dense2 = layers.Dense(1,
activation='sigmoid')

    def call(self, inputs):
        out1 = self.dense1(inputs)
        return self.dense2(out1)

# Instanciando o Modelo
neural_net = MyModel(name='MyModel')
# Aplicação Direta no Forward:
# pred = neural_net(X_test)
```

Não vamos empregar nesse modo

# Treinamento Customizado sem model.fit()

## O Loop de Otimização Explícito

Como funciona o Loop Interno de Treino?

1. Forward Pass: Passa o lote (batch_x) pelo modelo e calcula a loss_fn(batch_y, pred) dentro do GradientTape.

2. Obtenção de Gradientes: tape.gradient(loss, trainable_variables).

3. Atualização de Pesos: optimizer.apply_gradients(zip(gradients, trainable_variables)).

Relevância para LLMs:

- Permite implementar Loss Functions customizadas (como contraste, NCE ou perda de difusão) e acumulação de gradientes.

```
# Função de Otimização Customizada (T4)
def run_optimization(x, y):
    with tf.GradientTape() as tape:
        pred = neural_net(x)        # 1. Forward
        loss = loss_fn(y, pred)     # 2. Loss

    # 3. trainable_variables
    vars = neural_net.trainable_weights
    # 4. Gradientes via AutoGrad
    grads = tape.gradient(loss, vars)
    # 5. Atualiza W e b no Otimizador
    optimizer.apply_gradients(zip(grads, vars))

# Loop de Treino sobre Lotes (tf.data.Dataset)
for step, (batch_x, batch_y) in enumerate(train_data):
    run_optimization(batch_x, batch_y)
```

Não vamos empregar nesse modo

# Preparação de Dados e Subamostragem de Zipf

## Amostragem Ponderada e Performance

Tratamento de Palavras Muito Frequentes

- Palavras como 'the', 'of', 'and' fornecem pouca informação semântica.
- make_sampling_table(vocab_size) gera probabilidades inversas baseadas na Lei de Zipf para descartar stopwords ruidosas.

```
# Código de Preparação de Dados (TF_word2vec)
# Tabela de Subamostragem de Zipf
sampling_table =
tf.keras.preprocessing.sequence.make_sampling_table(vocab
_size)

# Amostragem de Negativos no TensorFlow
negative_sampling_candidates, _, _ =
tf.random.log_uniform_candidate_sampler(
    true_classes=context_class,
    num_true=1,
    num_sampled=num_ns,  # num_ns = 4
    unique=True,
    range_max=vocab_size,
    seed=SEED
)

# Estruturação da Base Final em Tensores
# targets.shape: (107193,)
# contexts.shape: (107193, 5)
# labels.shape: (107193, 5)
```

# Treinamento e Métricas do Word2Vec

## Evolução do Aprenzado em 20 Épocas

Configuração do Treino:

- Dimensão do Embedding: embedding_dim = 128.
- Otimizador: Adam.
- Loss: CategoricalCrossentropy(from_logits=True).
- Callback: TensorBoard(log_dir='logs').

Evolução Real das Métricas (Notebook):

- Época 01: Accuracy = 0.275, Loss = 1.600
- Época 05: Accuracy = 0.549, Loss = 1.224
- Época 10: Accuracy = 0.710, Loss = 0.902
- Época 20: Accuracy = 0.852, Loss = 0.527

```
# Compilação e Fit do Word2Vec (TF_word2vec)
embedding_dim = 128
word2vec = Word2Vec(vocab_size, embedding_dim)

word2vec.compile(
    optimizer='adam',

loss=tf.keras.losses.CategoricalCrossentropy(from_logits=
True),
    metrics=['accuracy']
)

tensorboard_callback =
tf.keras.callbacks.TensorBoard(log_dir='logs')

word2vec.fit(
    dataset,
    epochs=20,
    callbacks=[tensorboard_callback]
)
```

# Exportando e Visualizando os Embeddings

## Extração dos Pesos e Projetor 3D

Extração da Matriz de Pesos Treinada:

- Recupera a camada 'w2v_embedding' contendo a matriz de pesos (vocab_size, 128).

Arquivos TSV para Exportação:

- vectors.tsv: Contém os 128 valores numéricos de cada palavra por linha.
- metadata.tsv: Contém a palavra correspondente em texto plano.

TensorFlow Embedding Projector:

- Acesse projector.tensorflow.org.
- Carregue os dois arquivos TSV para explorar o espaço vetorial 3D interativo via PCA e t-SNE!

```
# Exportação para TSV (TF_word2vec)
# Extraindo pesos da camada de embedding
weights =
word2vec.get_layer('w2v_embedding').get_weights()[0]
vocab = vectorize_layer.get_vocabulary()

# Escrevendo arquivos TSV
out_v = io.open('vectors.tsv', 'w', encoding='utf-8')
out_m = io.open('metadata.tsv', 'w', encoding='utf-8')

for index, word in enumerate(vocab):
    if index == 0: continue  # pular padding
    vec = weights[index]
    out_v.write('\t'.join([str(x) for x in vec]) + '\n')
    out_m.write(word + '\n')
out_v.close()
out_m.close()
```

SÍNTESE & CONEXÕES

# Quadro Comparativo de Níveis de Abstração

| Dimensão | MLP Clássico (Scikit-Learn) | TensorFlow & Keras High-Level | TensorFlow Custom & Low-Level |
|---|---|---|---|
| **Nível de Abstração** | Altíssimo (Caixa Preta) | **Alto (Wrapper Produtivo)** | Baixo (Controle Absoluto) |
| **Representação** | Matrizes NumPy (CPU) | **Tensores multidimensionais (GPU)** | Tensores, AutoGrad e Grafos |
| **Flexibilidade** | Restrito a MLPs fixos | **Excelente para 90% das redes** | Total (Permite qualquer modelo) |
| **Aplicações em NLP** | Classificação de TF-IDF/BoW | **Classificação Sequencial/MLP** | Transformers, LLMs e Word2Vec |

CONCLUSÃO

# Síntese da Aula 3 e Ponte para Transformers

## Por que isso Habilita os Transformers?

• A Atenção nada mais é do que Operações com Tensores: O mecanismo de Multi-Head Attention (Q, K, V) é puramente multiplicação de matrizes (tf.matmul e tf.einsum).

• Treinamento de LLMs na Prática: O loop de treino com tf.GradientTape() e Adam é o exato padrão usado para ajustar modelos de bilhões de parâmetros.

REFERÊNCIAS

# Referências Bibliográficas e Materiais de Apoio

## Leituras recomendadas

1. Oliveira, Rogério de. Notebooks de Aula T1.ipynb, T2.ipynb, T3.ipynb e T4.ipynb. Trilha de Deep Learning, Inteligência Artificial. https://github.com/Rogerio-mack/Deep-Learning-I

LER PARA A PRÓXIMA AULA:

**2. WOLFRAM, Stephen. O que o ChatGPT faz... e por que funciona? Escritos de Stephen Wolfram, 2023.**